

Exercise Set 1

Problem 1

Task a

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("train.csv")
```

```
df.drop(columns=['id', 'partlybad'], inplace=True)
```

Task b

```
df[['T84.mean', 'UV_A.mean', 'CS.mean']].describe()
```

	T84.mean	UV_A.mean	CS.mean
count	450.000000	450.000000	450.000000
mean	6.439594	10.797330	0.003083
std	9.827282	6.626678	0.002246
min	-23.627710	0.438965	0.000343
25%	-0.733160	4.304971	0.001436
50%	7.661622	11.506937	0.002548
75%	14.118346	16.573188	0.004119
max	27.001804	22.500037	0.013706

Task c

```
arr = df['T84.mean'].values
mean = arr.mean()
std = arr.std()
print(f"Mean: {mean}, Std: {std}")
```

Mean: 6.439594405677285, Std: 9.816356767300478

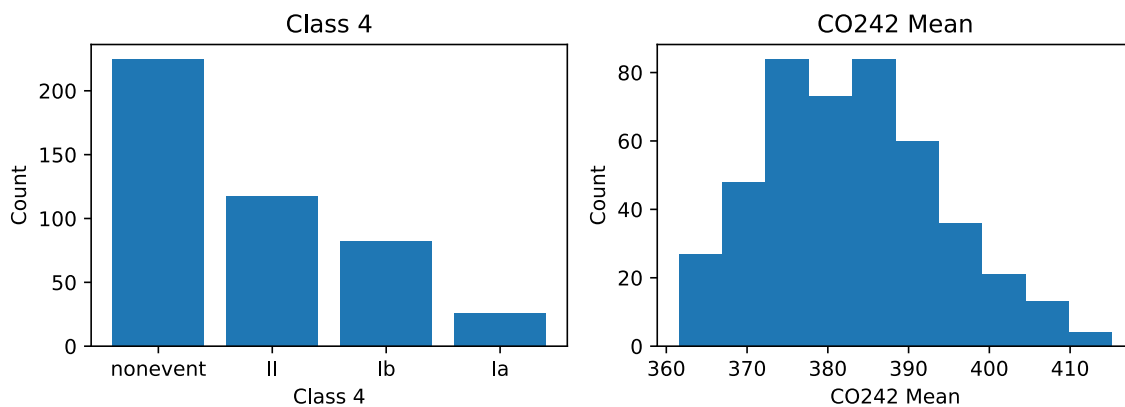
Task d

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))

axes[0].bar(df['class4'].value_counts().index,
df['class4'].value_counts().values)
axes[0].set_title('Class 4')
axes[0].set_xlabel('Class 4')
axes[0].set_ylabel('Count')

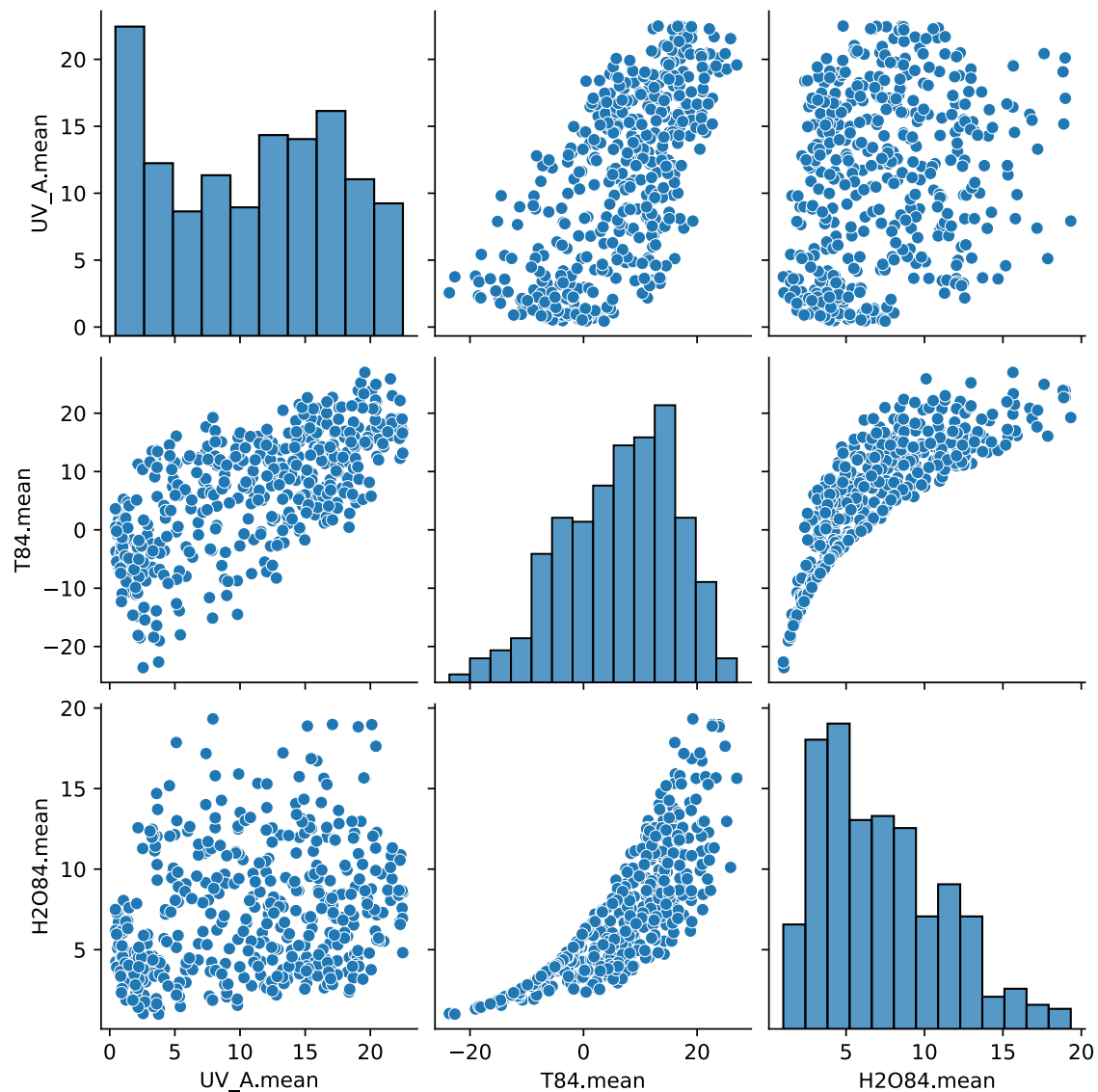
axes[1].hist(df['CO242.mean'])
axes[1].set_title('CO242 Mean')
axes[1].set_xlabel('CO242 Mean')
axes[1].set_ylabel('Count')

plt.tight_layout()
plt.show()
```



Task e

```
sns.pairplot(df[['UV_A.mean', 'T84.mean', 'H2084.mean']])
plt.show()
```



Task f

```
most_common_class = df['class4'].value_counts().idxmax()
p = (len(df) - df['class4'].value_counts()['nonevent']) / len(df)

test = pd.read_csv("test.csv")
submission = pd.DataFrame({'id': test['id'], 'class4': most_common_class, 'p':
p})
submission.to_csv("submission.csv", index=False)
```

Problem 2

Task a

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score
```

```
train_syn = pd.read_csv("train_syn.csv")
valid_syn = pd.read_csv("valid_syn.csv")
test_syn = pd.read_csv("test_syn.csv")
```

```
degrees = np.arange(0, 9)
train_loss = []
valid_loss = []
test_loss = []
test_trva_loss = []
cv_loss = []
```

```
X_train = train_syn.drop(columns=['y'])
y_train = train_syn['y']
X_valid = valid_syn.drop(columns=['y'])
y_valid = valid_syn['y']
X_test = test_syn.drop(columns=['y'])
y_test = test_syn['y']
X_trva = pd.concat([X_train, X_valid], axis=0)
y_trva = pd.concat([y_train, y_valid], axis=0)
```

```
def make_poly_ols(degree: int) -> Pipeline:
    include_bias = True if degree == 0 else False
    return Pipeline([
        ("poly", PolynomialFeatures(degree=degree,
    include_bias=include_bias)),
        ("scaler", StandardScaler()),
        ("ols", LinearRegression())
    ])
```

```
def compute_loss(X_train: pd.DataFrame, y_train: pd.Series, X_test:
pd.DataFrame, y_test: pd.Series) -> float:
    model = make_poly_ols(degree)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    return np.mean((y_test - y_pred) ** 2)
```

```
def cv_score_model(model: Pipeline, X: pd.DataFrame, y: pd.Series, cv: int =
```

```

10) -> float:
    kf = KFold(n_splits=cv, shuffle=True, random_state=42)
    scores = cross_val_score(model, X, y, scoring='neg_mean_squared_error',
cv=kf)
    return -scores.mean()

def compute_all_losses(degree: int):
    train_loss.append(compute_loss(X_train, y_train, X_train, y_train))
    valid_loss.append(compute_loss(X_train, y_train, X_valid, y_valid))
    test_loss.append(compute_loss(X_train, y_train, X_test, y_test))
    test_trva_loss.append(compute_loss(X_trva, y_trva, X_test, y_test))

    model = make_poly_ols(degree)
    model.fit(X_trva, y_trva)
    cv_loss.append(cv_score_model(model, X_trva, y_trva))

```

```

for degree in degrees:
    compute_all_losses(degree)

```

```

results_df = pd.DataFrame({
    'Degree': degrees,
    'Train': train_loss,
    'Valid': valid_loss,
    'Test': test_loss,
    'TRVA': test_trva_loss,
    'CV': cv_loss
}).set_index('Degree')
print(results_df)

```

	Train	Valid	Test	TRVA	CV
Degree					
0	18.459585	32.342363	22.100658	21.620221	28.779634
1	4.088535	7.127844	8.876307	9.934943	7.252893
2	0.218586	0.293732	0.245847	0.214016	0.321785
3	0.216819	0.283447	0.290080	0.275111	0.386635
4	0.118796	0.624726	0.969078	0.223993	0.641643
5	0.096532	0.573480	4.894836	1.039089	0.393977
6	0.007574	3.416787	213.297140	0.881471	0.399130
7	0.004999	6.862993	1261.988070	0.271719	1.738198
8	0.002082	401.651780	154266.871360	11.223554	25.777011